Module 2-2: Nonlinear Regression

Xiao Wang

Department of Statistics
Purdue University

Table of contents

1. Polynomial Regression

2. Polynomial Splines

3. Smoothing Splines

4. Local Regression

5. Regularization and Reproducing Kernel Hilbert Spaces

Polynomial Regression

Moving beyond linearity

- The truth is never linear!

Moving beyond linearity

- The truth is never linear!
- Or almost never!

Moving beyond linearity

- The truth is never linear!
- Or almost never!
- But often the linear assumption is good enough

Moving beyond linearity

- The truth is never linear!
- Or almost never!
- But often the linear assumption is good enough
- When it is not...
  — polynomials
  — step functions
  — splines
  — local regression
  — generalized additive models
  offer a lot flexibility, without losing the ease and interpretability of linear models.

Nonlinear regression models

Start with a dictionary of M basis functions:

- Polynomial: $h_m(\mathbf{x}) = x_j^2,\ x_j x_k, \ldots$
- Other nonlinear basis: $h_m(\mathbf{x}) = \| \mathbf{x} \|,\ \log(x_j), \ldots$
- Piecewise constant: $h_m(\mathbf{x}) = I(L_m < x_j < U_m)$

Model f with basis expansions:

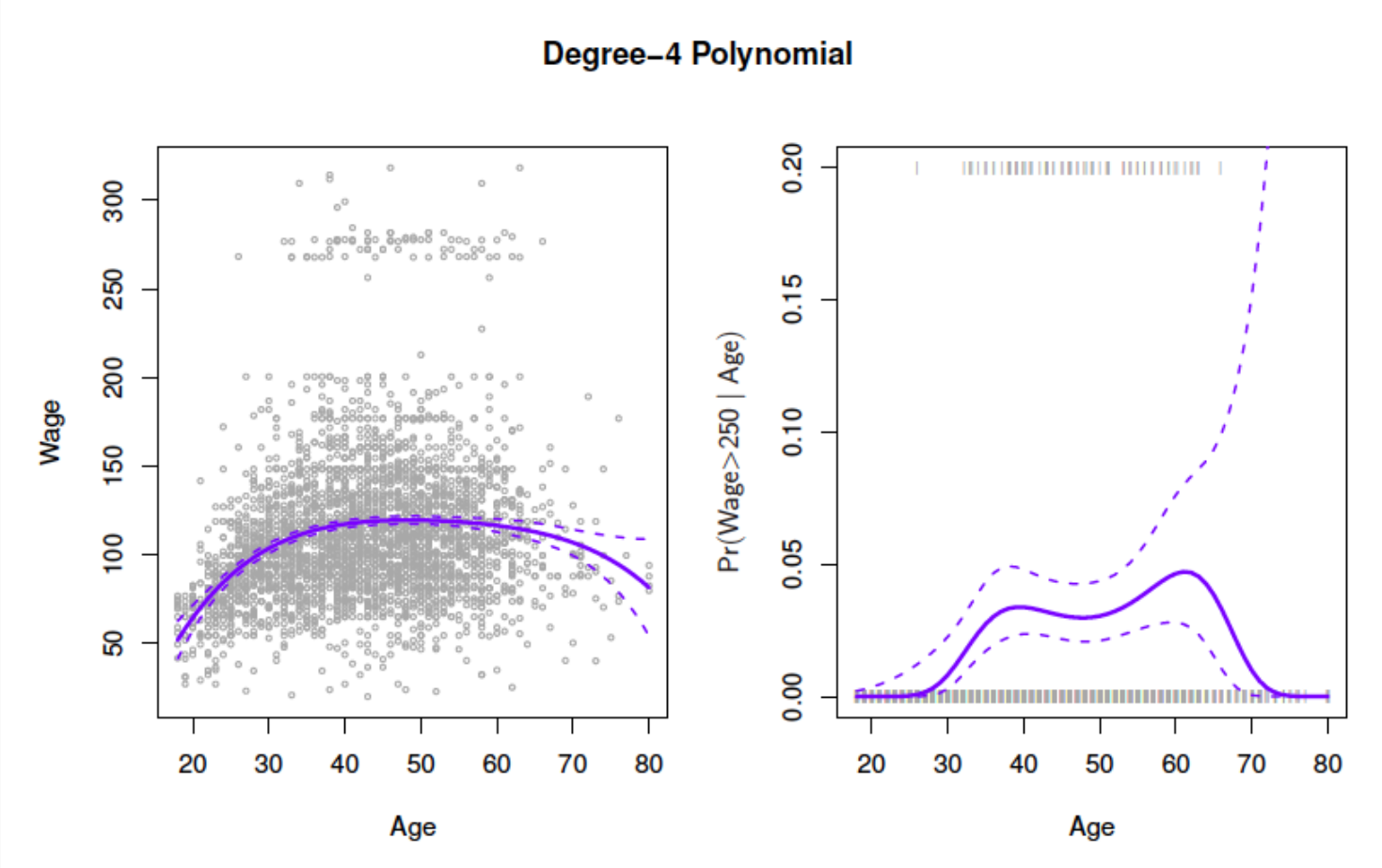
$$f(\mathbf{x}) = \sum_{m=1}^{M} \beta_m h_m(\mathbf{x})$$

Control the model complexity by:

- Restriction method
- Selection method
- Regularization method

Polynomial regression

$$y_i = \beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d + \epsilon_i$$



Details

- Create new variables $X_1 = X$, $X_2 = X^2$, etc and then treat as multiple linear regression

Details

- Create new variables $X_1 = X$, $X_2 = X^2$, etc and then treat as multiple linear regression
- Not really interested in the coefficients; more interested in the fitted function values at any value x_0:

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \cdots + \hat{\beta}_4 x_0^4$$

Details

- Create new variables $X_1 = X$, $X_2 = X^2$, etc and then treat as multiple linear regression
- Not really interested in the coefficients; more interested in the fitted function values at any value x_0:

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \cdots + \hat{\beta}_4 x_0^4$$

- Since $\hat{f}(x_0)$ is a linear function of the $\hat{\beta}_\ell$, can get a simple expression for pointwise-variances $\mathrm{var}(\hat{f}(x_0))$ at any value x_0. In the figure we have computed the fit and pointwise standard errors on a grid of values for x_0. We show $\hat{f}(x_0) \pm 2se(\hat{f}(x_0))$.

Details

- Create new variables $X_1 = X$, $X_2 = X^2$, etc and then treat as multiple linear regression
- Not really interested in the coefficients; more interested in the fitted function values at any value x_0:

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \cdots + \hat{\beta}_4 x_0^4$$

- Since $\hat{f}(x_0)$ is a linear function of the $\hat{\beta}_\ell$, can get a simple expression for pointwise-variances $\mathrm{var}(\hat{f}(x_0))$ at any value x_0. In the figure we have computed the fit and pointwise standard errors on a grid of values for x_0. We show $\hat{f}(x_0) \pm 2se(\hat{f}(x_0))$.
- We either fix the degree d at some reasonably low value, else use cross validation to choose d.

Details

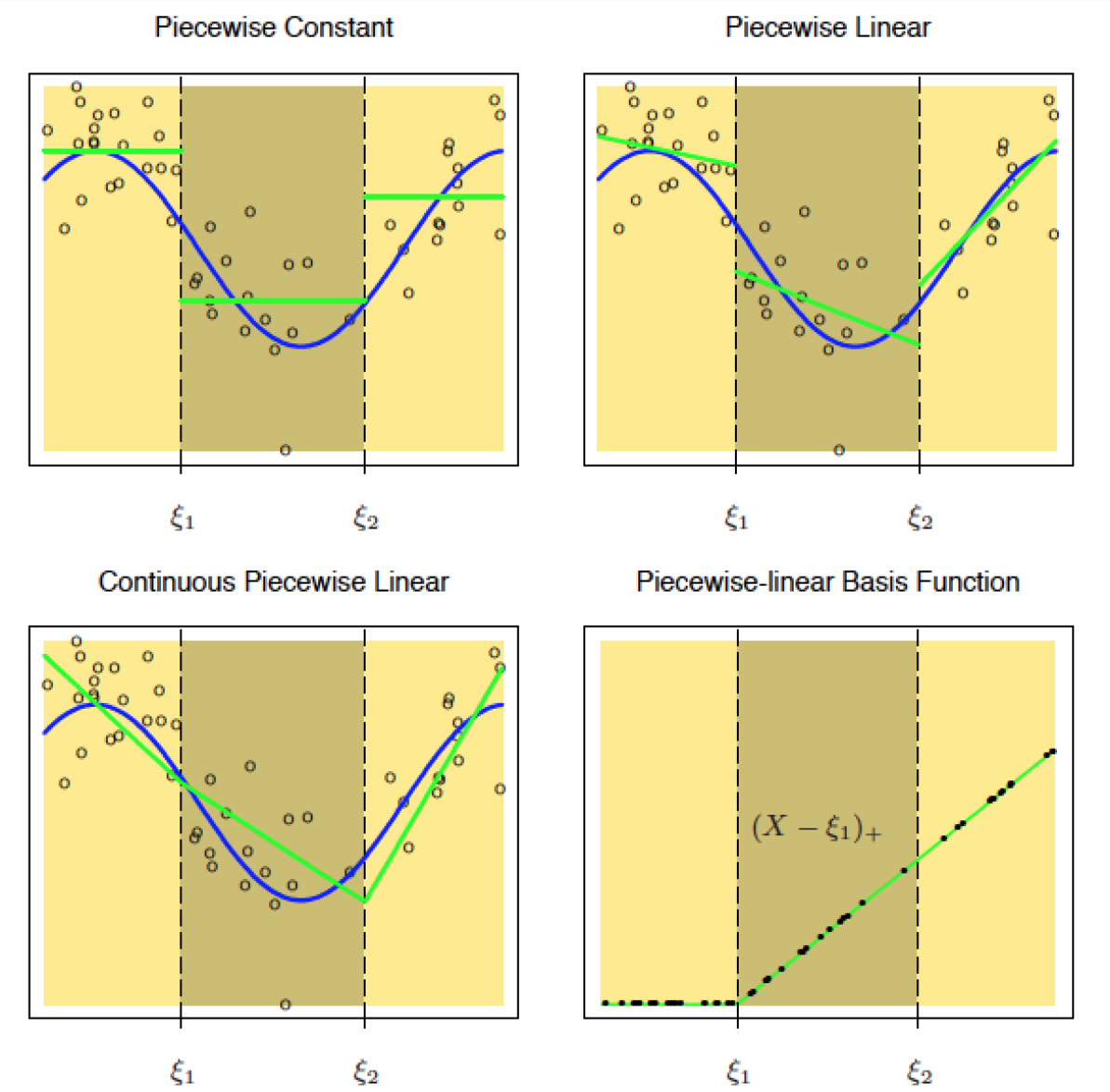
- Create new variables $X_1 = X$, $X_2 = X^2$, etc and then treat as multiple linear regression
- Not really interested in the coefficients; more interested in the fitted function values at any value x_0:

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \cdots + \hat{\beta}_4 x_0^4$$

- Since $\hat{f}(x_0)$ is a linear function of the $\hat{\beta}_\ell$, can get a simple expression for pointwise-variances $\mathrm{var}(\hat{f}(x_0))$ at any value x_0. In the figure we have computed the fit and pointwise standard errors on a grid of values for x_0. We show $\hat{f}(x_0) \pm 2se(\hat{f}(x_0))$.
- We either fix the degree d at some reasonably low value, else use cross validation to choose d.
- Caveat: polynomials have notorious tail behavior—very bad for extrapolation

Polynomial Splines

Piecewise linear



Piecewise linear

- In top left panel:
  $h_1(x) = I(x < \xi_1),\ h_2(x) = I(\xi_1 < x < \xi_2),\ h_3(x) = I(\xi_2 < x)$
- In top right panel, three additional basis: $h_{m+3}(x) = h_m(x)x$
- In bottom left panel, additional continuity restrictions are enforced:

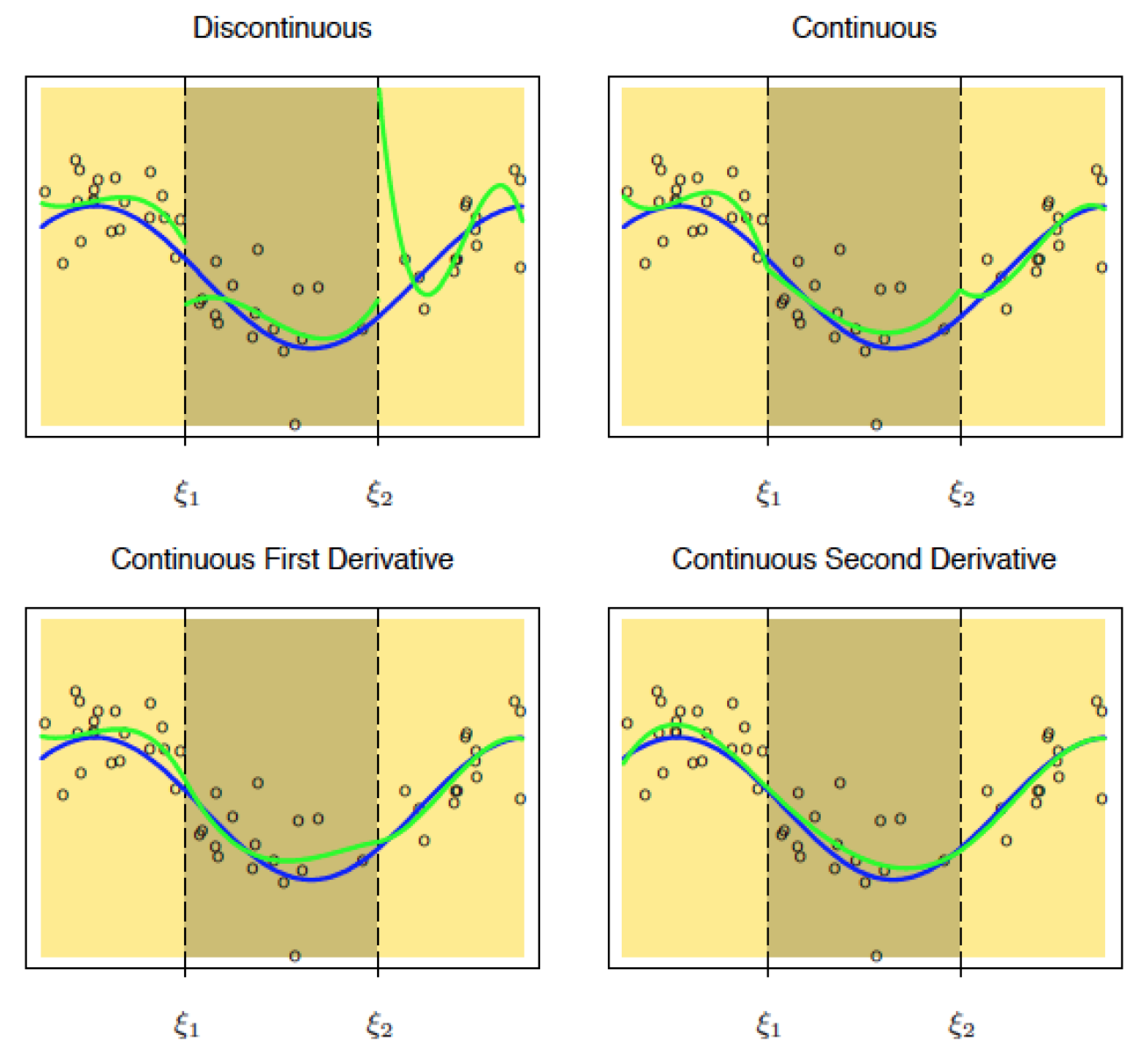
$$\begin{aligned} \beta_1 + \xi_1\beta_4 &= \beta_2 + \xi_1\beta_5 \\ \beta_2 + \xi_3\beta_5 &= \beta_3 + \xi_2\beta_6. \end{aligned}$$

- A more direct way to enforce continuity is to use

$$h_1(x) = 1,\ h_2(x) = x,\ h_3(x) = (x - \xi_1)_+,\ h_4(x) = (x - \xi_2)_+,$$

  where $z_+ = \max(0, z)$.

Piecewise polynomial



Piecewise polynomial and splines

- In bottom right panel, the function is continuous and has continuous first and second derivatives, known as *cubic spline*. It can be expanded by

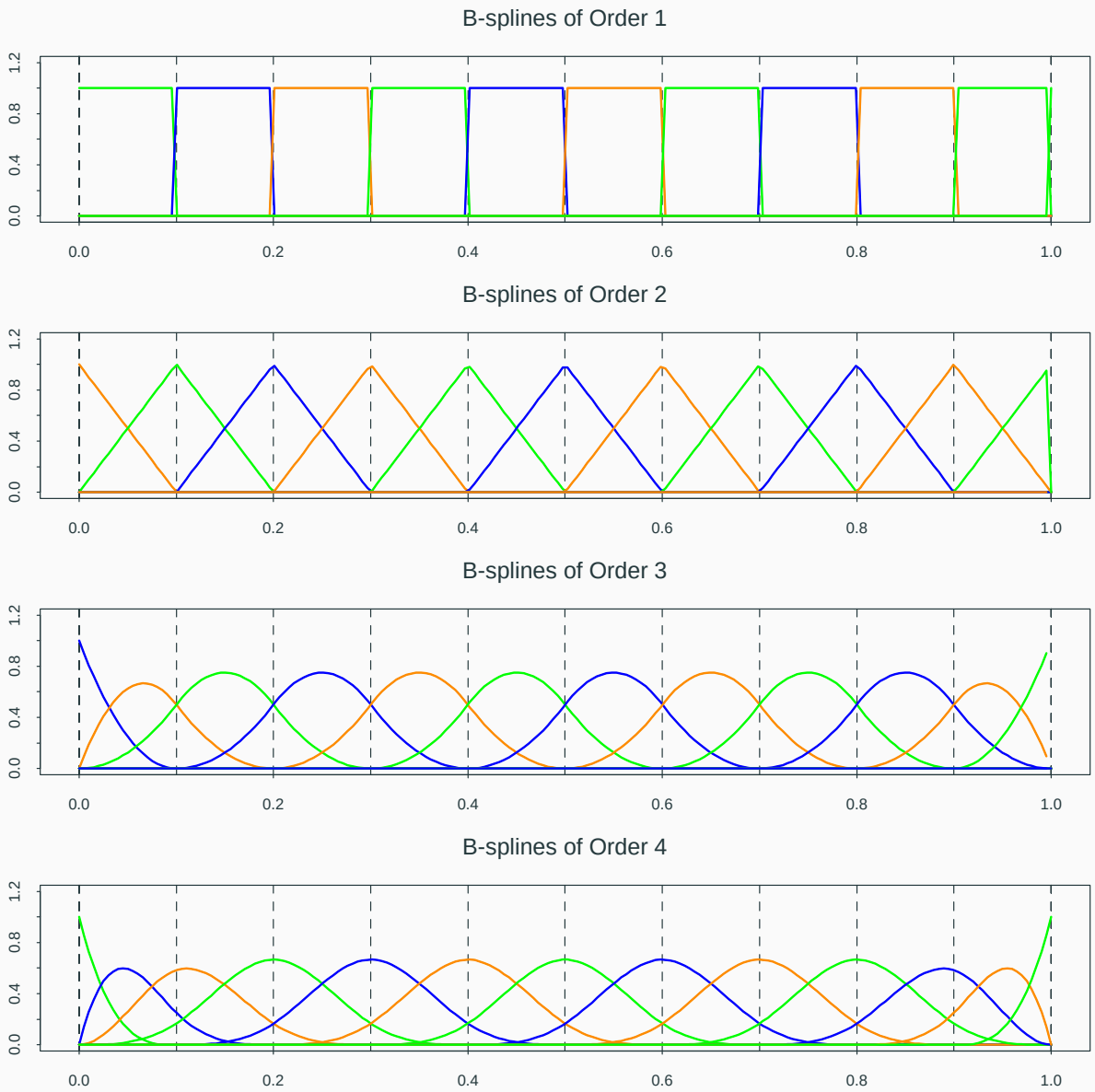
$$h_1(x) = 1,\ h_2(x) = x,\ h_3(x) = x^2,\ h_4(x) = x^3,$$
$$h_5(x) = (x - \xi_1)_+^3,\ h_6(x) = (x - \xi_2)_+^3.$$

- In general, an order-M spline with K knots ξ_j has continuous derivatives up to order $M - 2$, and can be expanded by:

$$h_j(x) = x^{j-1},\ j = 1, \ldots, M - 1$$
$$h_{M+l}(x) = (x - \xi_l)_+^{M-1},\ l = 1, \ldots, K.$$

- It is claimed that cubic splines are the lowest-order spline for which the knot-discontinuity is not visible to the human eyes.
- There is seldom any good reason to go beyond cubic-spline, unless one is interested in smooth derivatives.
- Other basis functions, such as B-spline.

B-spline basis functions



Natural cubic spline

- Since polynomial fit tends to be erratic near the boundaries, *natural cubic spline* adds additional constraint that the function is linear beyond the boundary knots.
- A natural cubic spline with K knots only requires K basis functions,

$$h_1(x) = 1, \; h_2(x) = x, \; h_{k+2}(x) = d_k(x) - d_{K-1}(x),$$

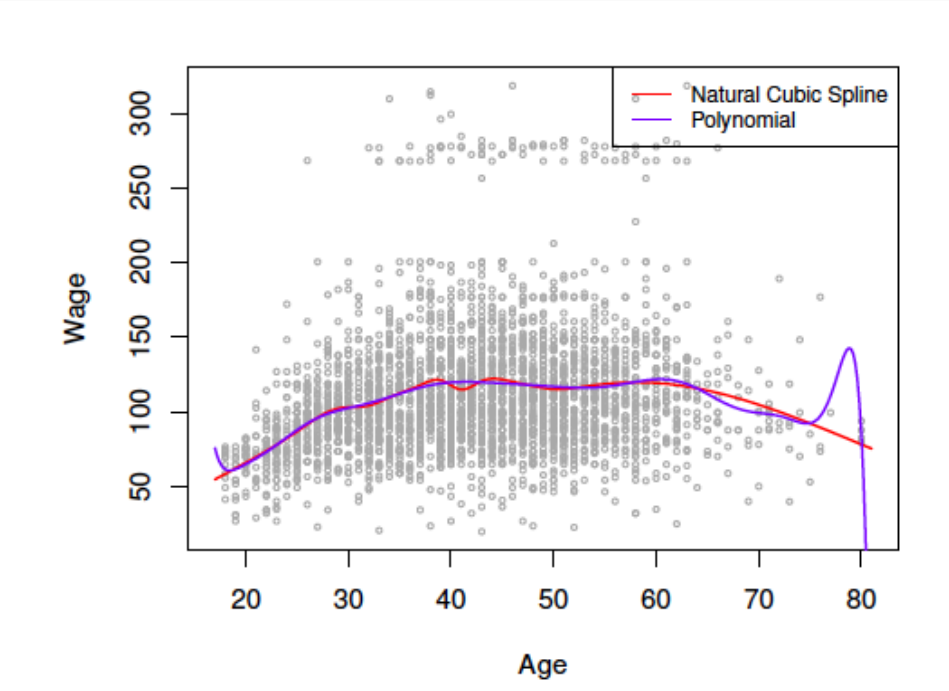
where

$$d_k(x) = \frac{(x - \xi_k)_+^3 - (x - \xi_K)_+^3}{\xi_K - \xi_k}.$$

One can verify that all basis functions are linear when $x > \xi_K$.

Knots placement

- One strategy is to decide K, the number of knots, and then place them at appropriate quantiles of the observed X.
- A cubic spline with K knots has $K + 4$ parameters or degrees of freedom.
- A natural spline with K knots has K degrees of freedom.



Comparison of a degree-14 polynomial and a natural cubic spline, each with 15df.

Smoothing Splines

Smoothing spline

⚠This section is a little bit mathematical! ⚠

$$\min_{f \in \mathcal{F}} \; RSS(f, \lambda) = \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int (f''(t))^2 dt$$

- $\mathcal{F}$ is a pre-specified functional class, e.g. all functions with continuous second derivatives.
- The first term is RSS, and tries to make $f(x)$ match the data at each x_i.
- The second term is a roughness penalty and controls how wiggly $f(x)$ is. It is modulated by the tuning parameter $\lambda \geq 0$.
- When $\lambda = 0$, solution interpolates data.
- When $\lambda = \infty$, solution is linear least squares fit.

- In general, the solution is a natural cubic spline with knots at each distinct x_i, $\hat{f}(x) = \sum_{j=1}^{n} N_j(x)\theta_j$.

- In general, the solution is a natural cubic spline with knots at each distinct x_i, $\hat{f}(x) = \sum_{j=1}^{n} N_j(x)\theta_j$.
- Smoothing splines avoid the knot-selection issue, leaving a single λ to be chosen.

- In general, the solution is a natural cubic spline with knots at each distinct x_i, $\hat{f}(x) = \sum_{j=1}^{n} N_j(x)\theta_j$.
- Smoothing splines avoid the knot-selection issue, leaving a single λ to be chosen.
- $RSS(f, \lambda) = (\mathbf{y} - \mathbf{N}\,\theta)^T(\mathbf{y} - \mathbf{N}\,\theta) + \lambda\theta\mathbf{\Omega}\theta$, $\mathbf{N}_{ij} = \mathbf{N}_j(x_i)$ and $\mathbf{\Omega}_{ij} = \int \mathbf{N}_i''(t)\mathbf{N}_j''(t)dt$.

- In general, the solution is a natural cubic spline with knots at each distinct x_i, $\hat{f}(x) = \sum_{j=1}^{n} N_j(x)\theta_j$.
- Smoothing splines avoid the knot-selection issue, leaving a single λ to be chosen.
- $RSS(f, \lambda) = (\mathbf{y} - \mathbf{N}\,\theta)^T(\mathbf{y} - \mathbf{N}\,\theta) + \lambda\theta\mathbf{\Omega}\theta$, $\mathbf{N}_{ij} = \mathbf{N}_j(x_i)$ and $\mathbf{\Omega}_{ij} = \int \mathbf{N}_i''(t)\mathbf{N}_j''(t)dt$.
- $\hat{\theta} = (\mathbf{N}^T\,\mathbf{N} + \lambda\mathbf{\Omega})^{-1}\mathbf{N}^T\,\mathbf{y}$.

Smoothing matrices

Smoothing spline is an example of *linear smoother*,

$$\hat{\mathbf{f}} = \mathbf{N}(\mathbf{N}^T\mathbf{N} + \lambda\mathbf{\Omega})^{-1}\mathbf{N}^T\mathbf{y} \equiv S_\lambda\,\mathbf{y}$$

- symmetric
- positive definite
- eigenvalues in $(0, 1]$, rank n
- $\mathrm{df}(\lambda)$ is defined to be $trace(S_\lambda)$
- Write $S_\lambda = (\mathbf{I} + \lambda K)^{-1}$, and $\hat{\mathbf{f}} = S_\lambda\,\mathbf{y}$ is the minimizer of

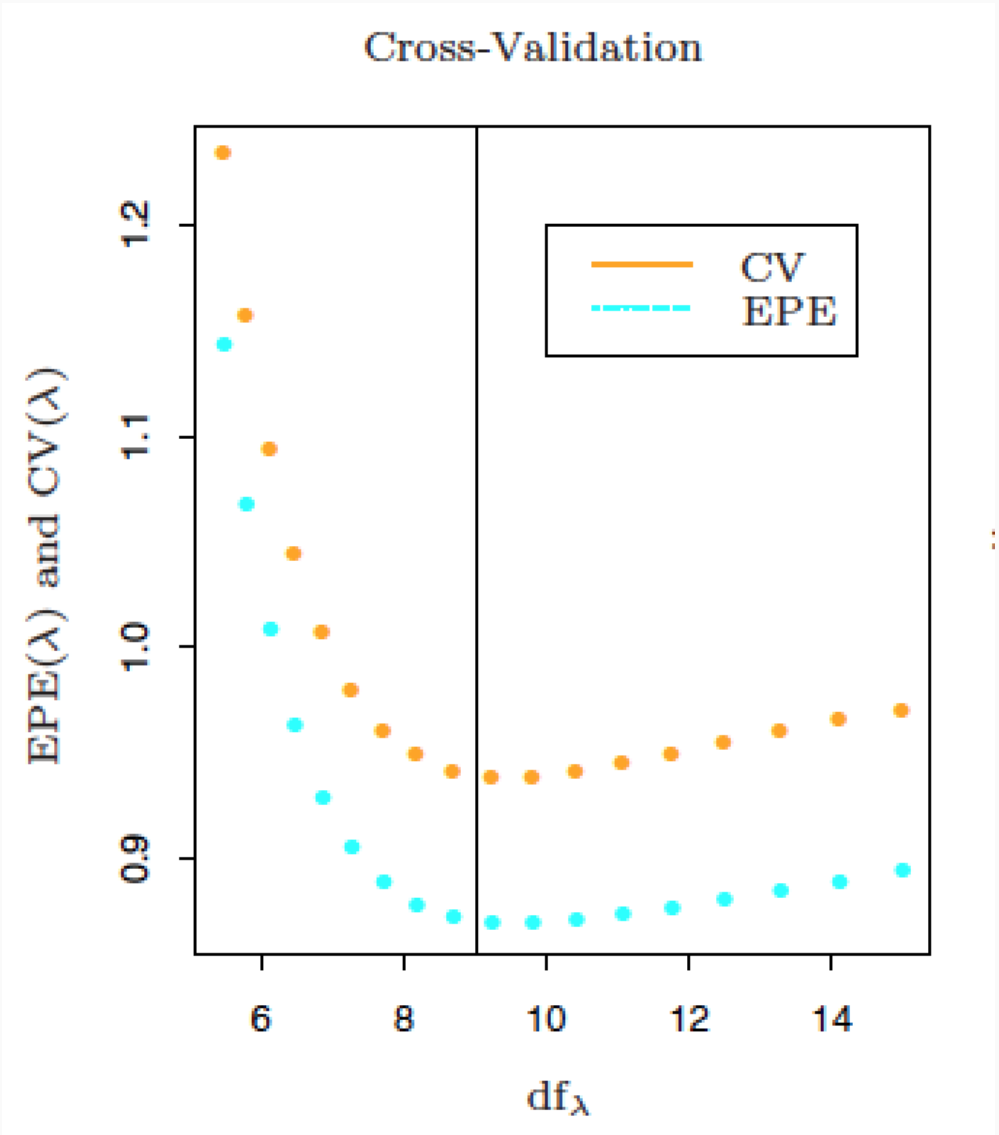
$$\|\,\mathbf{y} - \mathbf{f}\,\|^2 + \lambda\mathbf{f}^T K\,\mathbf{f}\,.$$

- $S_\lambda = \sum_{k=1}^{n} p_k(\lambda) u_k u_k^T$; $p_k(\lambda) = 1/(1 + \lambda d_k)$, where d_k is k-th eigenvalue of K and $\mathrm{df}(\lambda) = \sum p_k(\lambda)$.
- The *effective degrees of freedom* are given by

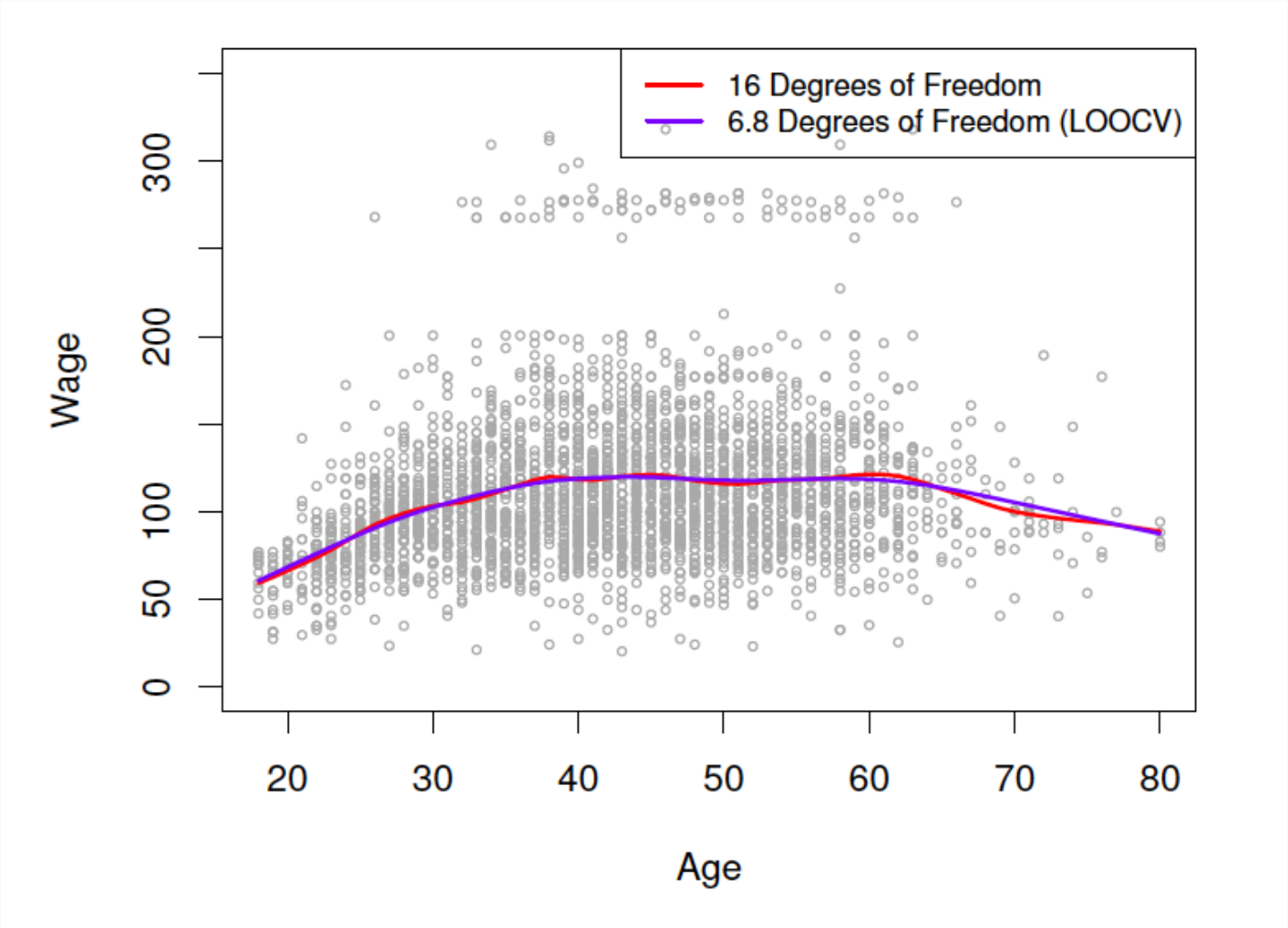
$$df_\lambda = \sum_{i=1}^{n} \{S_\lambda\}_{ii}$$

Tuning parameter selection: CV

As $\hat{\mathbf{f}}_\lambda = S_\lambda \mathbf{y}$, $\mathrm{CV}(\hat{\mathbf{f}}_\lambda) = \sum (y_i - \hat{f}_\lambda^{-i}(x_i))^2 = \frac{\sum (y_i - \hat{f}_\lambda(x_i))^2}{1-(S_\lambda)_{ii}}$



SS fit


16 Degrees of Freedom
6.8 Degrees of Freedom (LOOCV)
Wage
0
50
100
200
300
20
30
40
50
60
70
80
Age

Multidimensional splines

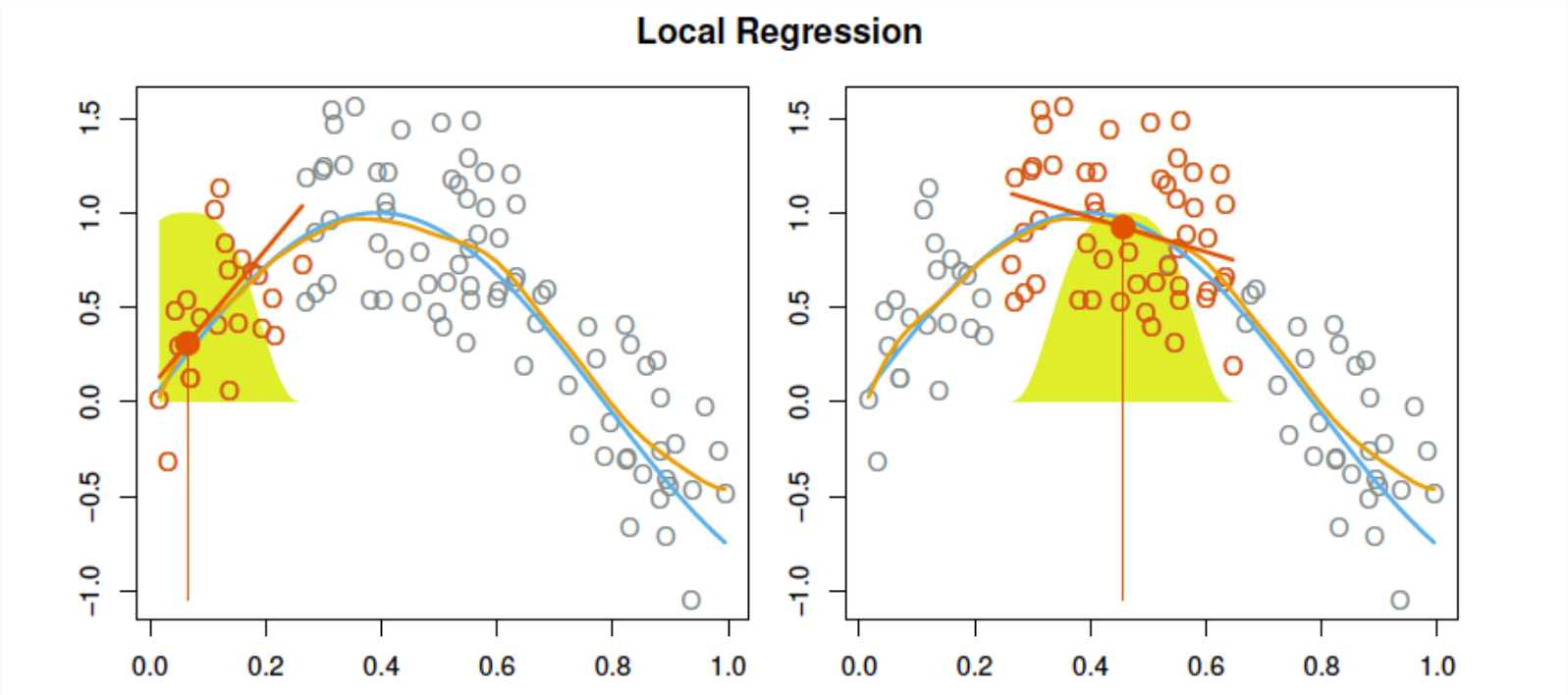
Assume $X \in \mathcal{R}^2$, $h_{1k}(x_1)$; $k = 1, \ldots, M_1$ are basis functions for x_1, and $h_{2l}(x_2)$; $l = 1, \ldots, M_2$ are basis functions for x_2.

- Additive basis: $h_{1k}(x_1)$, $h_{2l}(x_2)$
- Tensor product basis: $g_{kl}(x) = h_{1k}(x_1)h_{2l}(x_2)$
- Thin-plate spline uses the roughness penalty term

$$J(f) = \int\int \left(\left(\frac{\partial^2 f}{\partial x_1^2} \right)^2 + 2 \left(\frac{\partial^2 f}{\partial x_1 \partial x_2} \right)^2 + \left(\frac{\partial^2 f}{\partial x_2^2} \right)^2 \right) dx_1 dx_2$$

Local Regression

Local regression



With a sliding weight function, we fit separate linear fits over the range of X by weighted least squares.

Local kernel smoothing

- k-nearest neighbor estimation is a simple example of local kernel smoothing, where for each x_0, assign weight $w_i = I(x_i \in \mathcal{N}_k(x_0))$ to each x_i and estimate

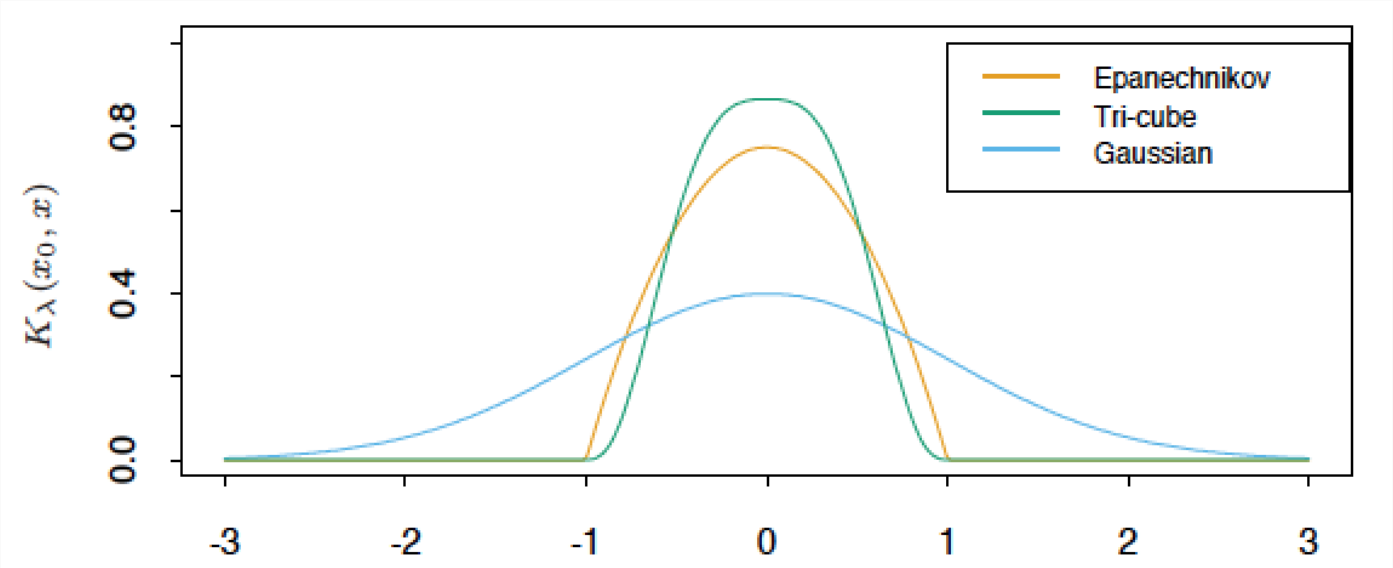
$$\hat{f}(\mathbf{x}_0) = \frac{\sum_{i=1}^n w_i y_i}{\sum_{i=1}^n w_i}.$$

- Nadaraya-Watson kernel-weighted estimation assigns a continuous weight to each x_i,

$$\hat{f}(x_0) = \frac{\sum_{i=1}^n K_\lambda(x_i, x_0) y_i}{\sum_{i=1}^n K_\lambda(x_i, x_0)},$$

where $K_\lambda(x, z) = D(|x - z|/\lambda)$, with the kernel width hyperparameter λ.

- Epanechnikov quadratic kernel: $D(t) = \frac{3}{4}(1 - t^2)I(|t| \leq 1)$
- Tri-cube kernel: $D(t) = (1 - |t|^3)^3 I(|t| \leq 1)$
- Guassian kernel: $D(t) = \exp(-t^2)$



Remarks and issues:

- The smoothing parameter λ needs to be properly determined
- λ can be replaced by $h_\lambda(x_0)$ as in k-NN.
- Boundary issues

Local linear regression

The Nadaraya-Watson estimation can be regarded as solution of

$$\min_{\alpha(x_0)} \sum_{i=1}^{n} K_\lambda(x_0, x_i)(y_i - \alpha(x_0))^2.$$

Local linear regression extends it to

$$\min_{\alpha(x_0),\beta(x_0)} \sum_{i=1}^{n} K_\lambda(x_0, x_i)(y_i - \alpha(x_0) - \beta(x_0)x_i)^2.$$

The estimate is then $\hat{f}(x_0) = \hat{\alpha}(x_0) + \hat{\beta}(x_0)x_0$. Its solution is

$$\hat{f}(x_0) = \mathbf{x}_0^T(\mathbf{X}^T\,\mathbf{W}(x_0)\,\mathbf{X})^{-1}\mathbf{X}^T\,\mathbf{W}(x_0)\,\mathbf{y} = \sum_{i=1}^{n} l_i(x_0)y_i,$$

where $\mathbf{x}_0 = (1, x_0)$, and $\mathbf{W}(x_0) = \text{diag}\{K_\lambda(x_i, x_0)\}$.

Local linear regression automatically corrects the bias to the first order, which needs additional treatment in the Nadaraya-Watson estimation.

$$
\begin{aligned}
& \mathbb{E}(\hat{f}(x_0)) = \sum_{i=1}^{n} l_i(x_0) f(x_i) \\
= \quad & \sum_{i=1}^{n} l_i(x_0) \Big(f(x_0) + f'(x_0)(x_i - x_0) + \frac{f''(x_0)}{2}(x_i - x_0)^2 + o((x_i - x_0)^2) \Big) \\
= \quad & f(x_0) + \sum_{i=1}^{n} l_i(x_0) \Big(\frac{f''(x_0)}{2}(x_i - x_0)^2 + o((x_i - x_0)^2) \Big),
\end{aligned}
$$

since $\sum_{i=1}^{n} l_i(x_0) = 1$ and $\sum_{i=1}^{n} l_i(x_0)(x_i - x_0) = 0$.

Multidimensional local regression

- Local smoothing extends naturally to multidimension

$$\min_{\alpha(\mathbf{x}_0),\beta(\mathbf{x}_0)} \sum_{i=1}^{n} K_\lambda(\mathbf{x}_0,\mathbf{x}_i)(y_i - \alpha(\mathbf{x}_0) - \beta(\mathbf{x}_0)^T\mathbf{x}_i)^2,$$

  with $K_\lambda(\mathbf{x}_0,\mathbf{x}) = D(\|\mathbf{x}_0 - \mathbf{x}\|/\lambda)$.
- When sample size is relatively small, structured local regression can be used
  - Structured kernel: $K_{\lambda,A} = D\Big((\mathbf{x}_0 - \mathbf{x})^T A(\mathbf{x}_0 - \mathbf{x})/\lambda\Big)$
  - Structured regression function

$$f(\mathbf{x}) = \alpha + \sum g_j(x_j) + \sum g_{kl}(x_k, x_l) + \cdots$$

  - Varying coefficient model

Regularization and Reproducing Kernel Hilbert Spaces

Reproducing kernel Hilbert spaces (RKHS)

⚠This section is a little bit mathematical! ⚠

- Suppose $h(x) : \mathbb{R}^p \to \mathbb{R}^M$, M large
- Given $x_1, \ldots, x_n$, let $\mathbf{H} = \{h_j(x_i)\}_{n \times M}$.
- $R(\beta) = (\mathbf{y} - \mathbf{H}\,\beta)^T(\mathbf{y} - \mathbf{H}\,\beta) + \lambda\beta^T\beta$
- Then

$$\begin{aligned} \hat{\mathbf{y}} &= \mathbf{H}\,\hat{\beta} \\ -\mathbf{H}^T(\mathbf{y} - \mathbf{H}\,\hat{\beta}) + \lambda\hat{\beta} &= 0 \\ -\mathbf{H}\,\mathbf{H}^T(\mathbf{y} - \mathbf{H}\,\hat{\beta}) + \lambda\,\mathbf{H}\,\hat{\beta} &= 0 \\ \hat{y} = \mathbf{H}\,\hat{\beta} &= (\mathbf{H}\,\mathbf{H}^T + \lambda\,\mathbf{I})^{-1}\,\mathbf{H}\,\mathbf{H}^T\,\mathbf{y} \end{aligned}$$

The interesting point is that $\hat{\mathbf{y}}$ only relies on $\mathbf{H}\,\mathbf{H}^T$ as opposed to $\mathbf{H}$ itself. Note that $\{\mathbf{H}\,\mathbf{H}^T\}_{i,i'} = \langle h(\mathbf{x}_i), h(\mathbf{x}_{i'})\rangle = K(\mathbf{x}_i, \mathbf{x}_{i'})$, and

$$\hat{f}(\mathbf{x}) = h(\mathbf{x})^T \hat{\beta} = \sum \hat{\alpha}_i K(\mathbf{x}, \mathbf{x}_i),$$

where $\hat{\alpha}_i = (\mathbf{K} + \lambda\,\mathbf{I})^{-1}\,\mathbf{y}$.

- Linear kernel: $K(x, z) = x^T z$
- Polynomial kernel: $K(x, z) = (1 + x^T z)^d$
- Radial (Gaussian) kernel: $K(x, z) = \exp(-\|x - z\|^2/2\sigma^2)$

Polynomial kernel

For example, assume $x \in \mathbb{R}^2$ and $d = 2$,

$$\begin{aligned} & K(x, z) = (1 + x_1 z_1 + x_2 z_2)^2 \\ = \quad & 1 + 2x_1 z_1 + 2x_2 z_2 + (x_1 z_1)^2 + (x_2 z_2)^2 + 2x_1 z_1 x_2 z_2 \end{aligned}$$

Then $M = 6$, and the corresponding basis functions are

$$\begin{aligned} & h_1(x) = 1, \ h_2(x) = \sqrt{2}x_1, \ h_3(x) = \sqrt{2}x_2 \\ & h_4(x) = x_1^2, \ h_5(x) = x_2^2, \ h_6(x) = \sqrt{2}x_1 x_2. \end{aligned}$$

Again, we don't need to know the basis functions!

RKHS

Assume K has a functional eigen-decomposition,

$$K(x, z) = \sum_{i=1}^{\infty} \gamma_i \phi_i(x) \phi_i(z),$$

where $\gamma_i \geq 0$ and $\sum \gamma_i^2 < \infty$.

Define a RKHS $\mathcal{H}_K$ as the set of $f(x) = \sum_{i=1}^{\infty} c_i \phi_i(x)$ with

$$\|f\|_{\mathcal{H}_K}^2 \equiv \sum_{i=1}^{\infty} c_i^2 / \gamma_i < \infty,$$

where $\|\cdot\|_{\mathcal{H}_K}$ is the norm induced by K.

Regularization on RKHS

Consider a general regularization framework

$$\min_{f \in \mathcal{F}} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i)) + \lambda J(f).$$

If we take $\mathcal{F} = \mathcal{H}_K$ and $J(f) = \|f\|^2_{\mathcal{H}_K}$, then it becomes

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i)) + \lambda \|f\|^2_{\mathcal{H}_K}.$$

Representor Theorem (Wahba, 1990) shows that the solution is

$$f(x) = \sum_{i=1}^{n} \alpha_i K(x, x_i),$$

which is finite dimensional!

Properties of $\mathcal{H}_K$

- $\langle K(\cdot, x_i), f \rangle_{\mathcal{H}_K} = f(x_i)$ (Reproducing property)
- $\langle K(\cdot, x_i), K(\cdot, x_j) \rangle_{\mathcal{H}_K} = K(x_i, x_j)$
- $J(f) = \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j K(x_i, x_j)$

The regularization problem can be rewritten as

$$\min_{\alpha \in \mathcal{R}^n} \sum_{i=1}^{n} L\Big(y_i, \sum_{j=1}^{n} \alpha_j K(x_i, x_j)\Big) + \lambda \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j K(x_i, x_j).$$

Additional readings:

Wavelet method: Section 5.9 of Hastie, Tibshirani, Friedman.

KERNEL METHODS IN MACHINE LEARNING, By Thomas Hofmann, Bernhard Scholkopf ¨ and Alexander J. Smola, Annals of Statistics, 2008.